



Buenos Aires
Ciudad

Buenos Aires
aprende

«Talento
Tech»

FRONT-END JS

**Clase n° 11 | Programación Modular con
Funciones**

¡Les damos la bienvenida!



 Vamos a comenzar a grabar la clase.

Índice

1

11. Programación modular con funciones

- Funciones: ¿Que son? Parámetros de entrada y de salida
- Scope global y local
- Programación modular vs. Funciones
- Ejercitación de funciones
- Parámetros.
- Funciones nativas.

2

12. DOM y eventos

- Manipulación del DOM
- Definición, alcance y su importancia para operar sobre elementos HTML
- Eventos en JS
- Eventos: ¿Que son, para qué sirven y cuáles son los más comunes?
- Escuchar un evento sobre el DOM

Introducción a las Funciones

Funciones

¿Qué son las Funciones?

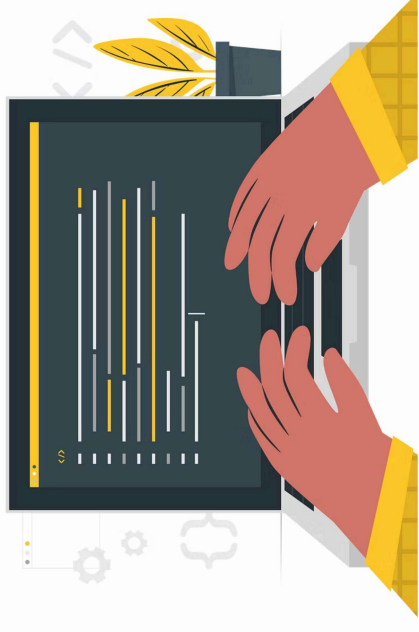
Son un conjunto de instrucciones que realizan tareas específicas.

Organizan el código

Permiten dividir programas complejos en partes más manejables.

Mejoran la legibilidad

Hacen que el código sea más fácil de entender y mantener.



Ejemplo de funciones:

Si tuvieras que escribir varias veces un código para calcular el área de un rectángulo, sería mejor crear una función que lo haga por vos

```
function calcularArea(largo, ancho) {  
  return largo * ancho;  
}  
  
console.log(calcularArea(5, 3)); // Devuelve 15
```

Parámetros de Entrada y Salida en Funciones

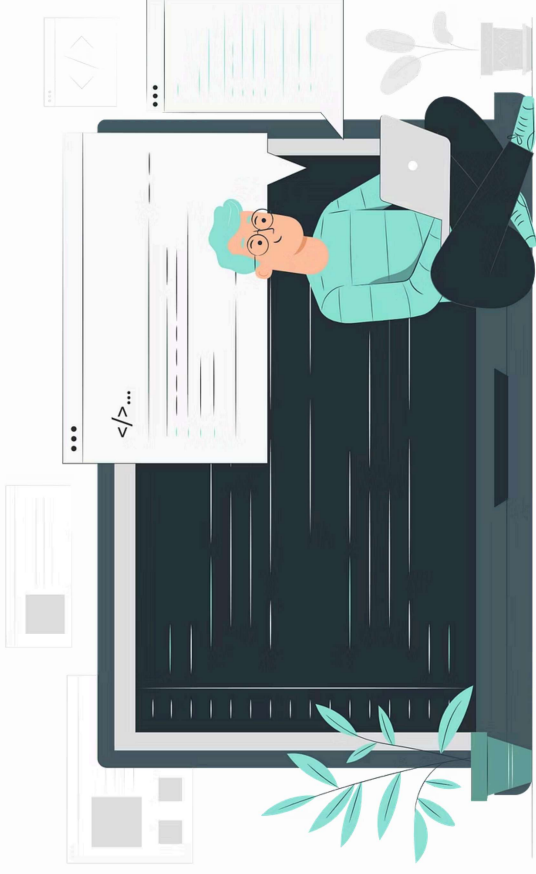
Parámetros de Entrada

Son variables que las funciones reciben para operar.



Flexibilidad

Permiten que las funciones trabajen con diferentes datos cada vez que se llaman.



Ejemplo:

Nombre es el parámetro de la función. Cuando la llamas, tenés que pasarle un argumento, que es el valor específico que querés usar:

```
function saludar(nombre) {  
  console.log("Hola " + nombre);  
}  
saludar("Juan"); // Esto imprimirá "Hola Juan"
```

Tipos de Funciones según Parámetros

Sin Parámetros

Función que no recibe ningún dato. Simplemente realiza una tarea y no necesita ninguna información externa.

- **Ejemplo:**

```
function saludar() {  
    console.log("Hola a todos");  
}  
saludar(); // Imprime: "Hola a todos"
```

Con Parámetros

Son más flexibles porque pueden trabajar con diferentes datos dependiendo de los argumentos que les pases

- **Ejemplo:**

```
function sumar(a, b) {  
    console.log(a + b);  
}  
sumar(5, 10); // Imprime: 15
```

Argumentos en Funciones

Los argumentos son los valores reales que pasamos a los parámetros de una función.

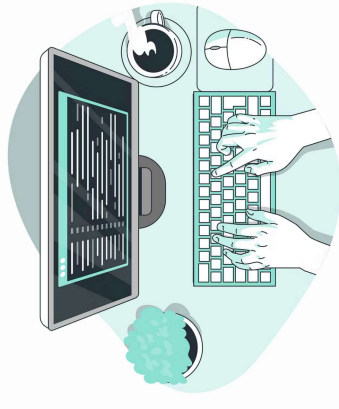
Uso

Si pensás en los parámetros como un recipiente, los argumentos son lo que llenan ese recipiente.

Ejemplo

Acá, 3 y 4 son los **argumentos** que reemplazan a a y b en la función. Al final, el resultado es $3 * 4$, que es 12.

```
function multiplicar(a, b) {  
    return a * b;  
}  
console.log(multiplicar(3, 4)); // Imprime: 12
```





Ejemplo con Múltiples Argumentos

Las funciones pueden recibir varios argumentos, y podés usarlos dentro de la función de la manera que prefieras.

```
function presentar(nombre, edad, ciudad) {  
  console.log(nombre + " tiene " + edad + " años y  
vive en " + ciudad + ".");  
}  
  
presentar("Juan", 25, "Buenos Aires");  
  
// Imprime: "Juan tiene 25 años y vive en Buenos  
Aires."
```

Scope Global y Local en Funciones

Alcance de Variables (Scope)

Scope Local

Variables solo accesibles dentro de la función donde se declaran.

Ejemplo: Aquí, `nombre` está definida dentro de la función `saludar`, por lo que solo es accesible dentro de esa función.

```
function saludar() {  
  let nombre = "Maria";  
  console.log("Hola " + nombre);  
}  
  
saludar(); // Imprime: "Hola Maria"  
console.log(nombre); // Error, 'nombre' no está definida fuera de la función
```

Alcance de Variables (scope)

Uno de los conceptos más importantes a la hora de trabajar con funciones es el **alcance** o **scope**. Define dónde y cuándo una variable es accesible dentro del código. Hay dos tipos principales: **global** y **local**.

Scope Global

Variables accesibles desde cualquier parte del programa. Se declaran fuera de las funciones.

Ejemplo: En este caso, **nombre** es global, porque está definida fuera de la función **saludar**.

```
let nombre = "Pedro";  
function saludar() {  
  console.log("Hola " + nombre);  
}  
saludar(); // Accede a la variable global  
'nombre' y muestra: "Hola Pedro"
```



El **scope global** puede ser riesgoso porque cualquier función o parte del programa puede modificar sus variables. Por ello, se recomienda priorizar el uso de **variables locales** para evitar errores y mantener el control del código.

Programación Modular vs. Funciones

Programación Modular

Enfoque que divide el código en módulos o partes pequeñas e independientes.

1

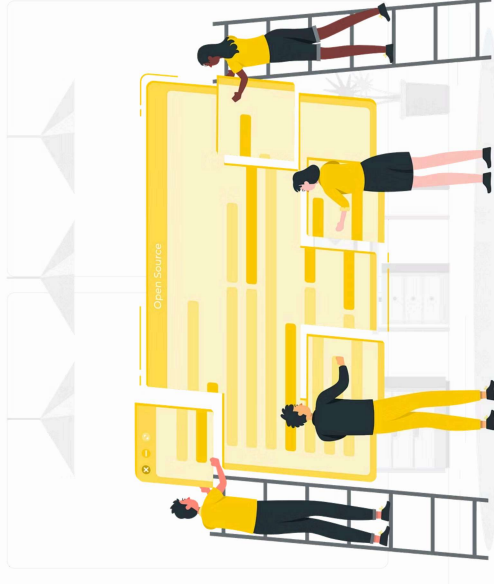
Ventaja Principal

Facilita la reutilización y el mantenimiento del código.

2

Relación con Funciones

Las **funciones** son la base de esta modularidad. A través de ellas, se puede organizar el código de manera más clara, donde cada función se ocupa de una tarea específica



Ventajas de la Programación Modular

1

Mantenimiento simplificado

Modificar una función afecta solo a esa parte del código.

2

Reutilización

Las mismas funciones pueden usarse en diferentes partes del programa.

3

Legibilidad

Cada función tiene un propósito claro, mejorando la comprensión del código.

Ejemplo

Cada función tiene una tarea específica: una calcula el área y otra imprime el resultado. Este enfoque hace que el código sea mucho más fácil de manejar.

```
function calcularAreaRectangulo(base, altura) {  
    return base * altura;  
}  
  
function imprimirResultado(area) {  
    console.log("El área del rectángulo es: " + area);  
}  
  
let base = 5;  
let altura = 10;  
let area = calcularAreaRectangulo(base, altura);  
imprimirResultado(area);
```

Funciones Nativas de JavaScript

Funciones Nativas de JavaScript

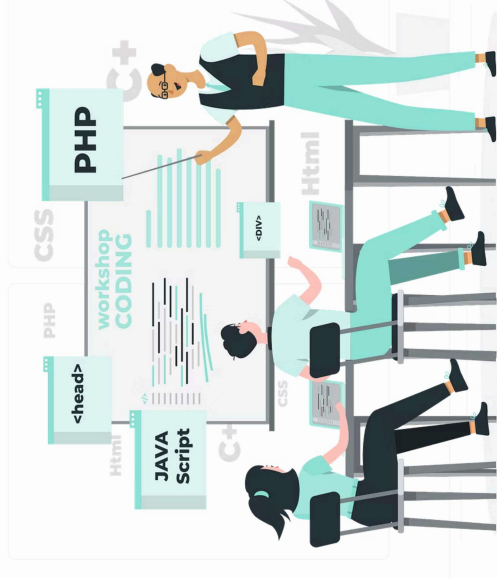
Funciones predefinidas en JavaScript, listas para usar.

Ventaja

Resuelven problemas comunes sin necesidad de escribir código adicional.

Eficiencia

Se pueden usar sin necesidad de definir las, ya que están incluidas en el entorno de ejecución.



Ejemplos de Funciones Nativas

alert()

Muestra un cuadro de alerta con un mensaje.

```
alert("¡Bienvenido a mi sitio web!");
```

parseInt()

Convierte un string en un número entero.

```
let numero = parseInt("123");  
console.log(numero); // Imprime 123
```

Date()

Devuelve la fecha y hora actual.

```
let fechaActual = new Date();  
console.log(fechaActual);
```

Math.random()

Genera un número aleatorio entre 0 y 1.

```
let aleatorio = Math.random();  
console.log(aleatorio);
```

Más Funciones Nativas Útiles

console.log()

Imprime mensajes en la consola del navegador.

```
console.log("Este es un mensaje para la consola");
```

toUpperCase() y toLowerCase()

Convierte un string a mayúsculas o minúsculas.

```
let texto = "Hola Mundo";  
console.log(texto.toUpperCase()); // Imprime "HOLA MUNDO"
```

```
console.log(texto.toLowerCase()); // Imprime "hola mundo"
```

slice()

Extrae una parte de un string o array.

```
let frase = "JavaScript es increíble";  
let parte = frase.slice(0, 10);  
console.log(parte); // Imprime "JavaScript"
```

¿Por qué usar funciones nativas?

1

Eficiencia

Están Optimizadas, son más rápidas y eficientes que escribir tu propia versión de esas funciones.

2

Simplicidad

Simplifican el código y lo hacen más legible.

3 Menos Errores

Probadas extensivamente, reduciendo errores en el código.

Ejemplo Práctico con Funciones Nativas

Acá estamos usando `prompt()` para pedir el nombre, `toUpperCase()` para convertirlo a mayúsculas, y `alert()` para mostrar el saludo. Además, con `console.log()` registramos el nombre ingresado en la consola.

```
let nombre = prompt("¿Cuál es tu nombre?");
if (nombre) {
  let saludo = "Hola, " + nombre.toUpperCase() + "!";
  alert(saludo);
  console.log("El usuario se llama " + nombre);}
```

Funciones vs. Programación Modular

Funciones

- Bloques de código para tareas específicas.
- Las funciones pueden definirse de manera que resuelvan una acción concreta (como sumar dos números o mostrar un mensaje).

Programación Modular

- Implica dividir todo el programa en módulos (que pueden contener varias funciones).
- Cada módulo es responsable de una parte del comportamiento del programa. Estos módulos son reutilizables y, generalmente, son independientes entre sí.

Cuándo Usar Funciones vs. Programación Modular

Funciones

Para tareas únicas y específicas que se pueden encapsular fácilmente.

Programación Modular

En proyectos grandes o complejos que requieren una estructura organizativa más amplia.



Carrito de compras

Ejemplo: Proyecto de Carrito de Compras

- ✓ Vamos a suponer que estás creando una pequeña aplicación que maneja un carrito de compras en una tienda online.



Paso 1: Crear funciones individuales para manejar las tareas.

```
function agregarProducto(producto) {  
  console.log(producto + " fue agregado al carrito.");  
}  
function calcularTotal(productos) {  
  let total = productos.reduce((sum, producto) => sum + producto.precio, 0);  
  console.log("El total es: $" + total);  
  return total;  
}
```

Paso 2: Modularizar el código.

Podrías tener un módulo separado para el carrito de compras, otro para la interfaz de usuario, y otro para el cálculo de precios.

```
// Módulo del carrito de compras
const carrito = (function() {
  let productos = [];
  return {
    agregar: function(producto) {
      productos.push(producto);
      agregarProducto(producto.nombre);
    },
    total: function() {
      return calcularTotal(productos);
    }
  };
})();
```

Ejercicio Práctico #1

Optativo | No entregable

Función para Validar la Edad de una Persona

Pasos a seguir:

Crear una función que reciba como parámetros el nombre y la edad de una persona. La función debe realizar lo siguiente:

Tips claves:

- **Uso de parámetros:** Pasar el nombre y la edad como argumentos a la función.
- **Condicionales:** Usar un `if...else` para determinar si es mayor o menor de edad.

1. Verificar si la persona es mayor de edad (18 años o más).
 - **Consola del navegador:** Utilizá `console.log()` para mostrar los resultados en la consola.
2. Mostrar un mensaje en la consola que diga si la persona es mayor o menor de edad.
3. Si es menor de edad, mostrar también cuántos años le faltan para cumplir 18

Ejercicio Práctico #2

Optativo | No entregable

Función para Calcular el Precio Total de un Producto con IVA

Pasos a seguir:

Crear una función que reciba el precio de un producto y el porcentaje de IVA (Impuesto al Valor Agregado). La función debe:

1. Calcular el precio total del producto, sumando el IVA.
2. Mostrar el precio total en la consola.
3. Hacer que el IVA sea un parámetro opcional, con un valor predeterminado del 21% (típico en Argentina).

Tips claves:

- **Parámetros opcionales:** Definir un valor por defecto para el IVA si no se proporciona uno.
- **Operaciones matemáticas:** Calcular el IVA y sumarlo al precio del producto.
- **Consola del navegador:** Mostrar el precio final con `console.log()`.

